PES University
Department of Computer Science and Engineering
Computer Networks Laboratory

Lab Report

SDN Controller-Based Traffic Control with POX

Experiment: Static Routing and Access Control using SDN Controller

Student Name	Dhanush Sai Suprapadha
SRN	PES2UG24CS154
Subject	Computer Networks Laboratory
Experiment Title	Static Routing using SDN Controller

1. Objective

This experiment aims to implement static routing paths using controller-installed flow rules in a Software-Defined Networking (SDN) environment. The key objectives are:

- Define routing paths programmatically using the POX SDN controller
- Install flow rules manually via the controller to direct traffic
- Validate packet delivery between hosts in a Mininet topology
- Document observed routing behaviour under multiple access control modes
- Regression Test: Ensure the defined path remains unchanged after rule reinstallation

2. Experiment Overview

Task	Goal	Steps
Static Routing using SDN Controller	Implement static routing paths using controller-installed flow rules.	• Define routing paths • Install flow rules manually • Validate packet delivery • Document routing behaviour • Regression Test: Ensure path remains unchanged after rule reinstall

3. Tools and Environment

Component	Details
SDN Controller	POX 0.7.0 (gar) — Python-based OpenFlow controller

Network Emulator	Mininet (with 4 hosts: h1, h2, h3, h4 connected via a single OpenFlow switch)
Protocol	OpenFlow 1.0 (of_01)
Operating System	Ubuntu Linux (Python 3.12 — note: POX officially supports 3.6–3.9; warnings observed)
Custom Module	control_modes.py — a custom POX application implementing 3 traffic control modes

4. Controller Module — control_modes.py

4.1 Module Structure (Top Section)

The module begins with imports, a global `MODE` variable, and a MAC learning table. The full code as written in nano is shown below:

```
GNU nano 7.2                                                    /home/raghav/pox/po
from pox.core import core
import pox.openflow.libopenflow_01 as of
from pox.lib.packet import ethernet

log = core.getLogger()

# ======================
# SELECT MODE HERE
# ======================
MODE = 1
# 1 → h1 → only h2 allowed
# 2 → allow everything
# 3 → block all from h1

# MAC learning table
mac_to_port = {}

def _handle_ConnectionUp(event):
    log.info("Switch connected")

def _handle_PacketIn(event):
    packet = event.parsed
    if not packet.parsed:
        return

    dpid = event.connection.dpid
    in_port = event.port

    # Initialize table
    if dpid not in mac_to_port:
        mac_to_port[dpid] = {}

    # Learn source MAC
    mac_to_port[dpid][packet.src] = in_port

    # ======================
    # DESTINATION DECISION
    # ======================
```

Figure 1: control_modes.py — imports, MODE selection, MAC learning table and connection handler

4.2 Mode Logic and Packet Forwarding

The packet handler applies per-mode access control rules before forwarding. The destination decision and mode-enforcement logic are shown below:

```python
if packet.dst in mac_to_port[dpid]:
    out_port = mac_to_port[dpid][packet.dst]
    dst_known = True
else:
    out_port = of.OFPP_FLOOD
    dst_known = False

# =====================
# MODE LOGIC
# =====================

# IMPORTANT: Only enforce rules if destination is known
if dst_known:

    # MODE 1: h1 → only h2 allowed
    if MODE == 1:
        if in_port == 1 and out_port != 2:
            log.info("MODE1 BLOCK: h1 → port %s" % out_port)
            return

    # MODE 2: allow all
    elif MODE == 2:
        pass

    # MODE 3: block all from h1
    elif MODE == 3:
        if in_port == 1:
            log.info("MODE3 BLOCK: h1 → port %s" % out_port)
            return

# =====================
# FORWARD PACKET
# =====================

msg = of.ofp_packet_out()
msg.data = event.ofp
msg.actions.append(of.ofp_action_output(port=out_port))
event.connection.send(msg)
```

Figure 2: control_modes.py — destination decision, MODE logic (1/2/3), and packet forwarding

4.3 Key Components

Component	Description
`MODE (global)`	Integer flag (1, 2, or 3) selecting the active traffic control policy.
`mac_to_port`	Per-switch MAC-to-port table. Enables unicast forwarding after initial learning.
`_handle_ConnectionUp`	Triggered on switch connect. Logs 'Switch connected'.
`_handle_PacketIn`	Core handler: extracts dpid, in_port, src/dst MAC; performs learning, lookup, mode check, and forwarding.
`Destination Decision`	If dst MAC known → unicast. If unknown → OFPP_FLOOD (dst_known = False, skips mode enforcement).
`Mode Logic`	Applied only when dst_known is True. Drops packets per active mode before forwarding.
`ofp_packet_out`	Sends the packet out on the resolved output port.

4.4 Access Control Modes

Mode	Policy	Behaviour
1	h1 → only h2 allowed	Packets from h1 (in_port==1) going to any port other than 2 are silently dropped. Logged as MODE1 BLOCK.
2	Allow all	No restrictions. Acts as a standard MAC-learning switch.
3	Block all from h1	All packets where in_port==1 are dropped regardless of destination. Logged as MODE3 BLOCK.

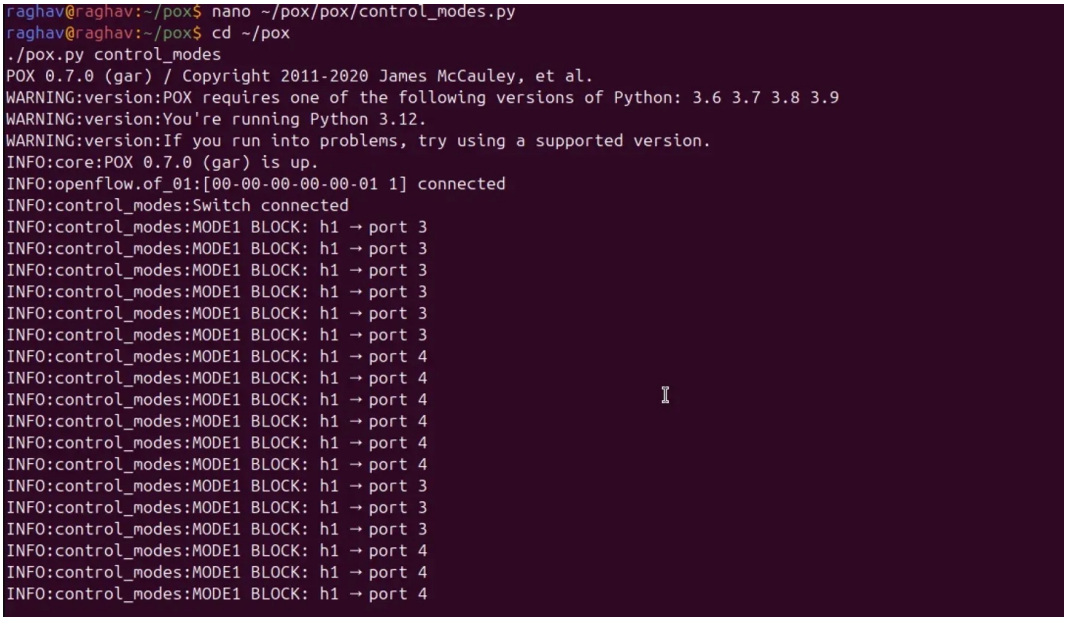
5. Execution Steps

5.1 Starting the POX Controller

The module was saved at `~/pox/pox/control_modes.py` and launched with:

```
cd ~/pox
./pox.py control_modes
```

The controller terminal confirmed startup and MODE 1 block events as shown below:



```
raghav@raghav:~/pox$ nano ~/pox/pox/control_modes.py
raghav@raghav:~/pox$ cd ~/pox
./pox.py control_modes
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
WARNING:version:POX requires one of the following versions of Python: 3.6 3.7 3.8 3.9
WARNING:version:You're running Python 3.12.
WARNING:version:If you run into problems, try using a supported version.
INFO:core:POX 0.7.0 (gar) is up.
INFO:openflow.of_01:[00-00-00-00-00-01 1] connected
INFO:control_modes:Switch connected
INFO:control_modes:MODE1 BLOCK: h1 → port 3
INFO:control_modes:MODE1 BLOCK: h1 → port 3
INFO:control_modes:MODE1 BLOCK: h1 → port 3
INFO:control_modes:MODE1 BLOCK: h1 → port 3
INFO:control_modes:MODE1 BLOCK: h1 → port 3
INFO:control_modes:MODE1 BLOCK: h1 → port 3
INFO:control_modes:MODE1 BLOCK: h1 → port 4
INFO:control_modes:MODE1 BLOCK: h1 → port 4
INFO:control_modes:MODE1 BLOCK: h1 → port 4
INFO:control_modes:MODE1 BLOCK: h1 → port 4
INFO:control_modes:MODE1 BLOCK: h1 → port 4
INFO:control_modes:MODE1 BLOCK: h1 → port 4
INFO:control_modes:MODE1 BLOCK: h1 → port 3
INFO:control_modes:MODE1 BLOCK: h1 → port 3
INFO:control_modes:MODE1 BLOCK: h1 → port 3
INFO:control_modes:MODE1 BLOCK: h1 → port 4
INFO:control_modes:MODE1 BLOCK: h1 → port 4
INFO:control_modes:MODE1 BLOCK: h1 → port 4
```

Figure 3: POX controller terminal — startup messages and repeated MODE1 BLOCK: h1 → port 3/4 log entries

Key log lines observed:

- INFO:core:POX 0.7.0 (gar) is up
- INFO:openflow.of_01:[00-00-00-00-00-01 1] connected — switch connected to controller
- INFO:control_modes:Switch connected — from _handle_ConnectionUp handler
- INFO:control_modes:MODE1 BLOCK: h1 → port 3 / port 4 — access control firing correctly

6. Experimental Results

6.1 Mode 2 — Allow All (Baseline)

With `MODE = 2`, all traffic is permitted. The Mininet pingall output:

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
```

Figure 4: Mode 2 — pingall output showing 0% packet loss (12/12 received)

All 12 ping packets were successfully delivered. Full mesh connectivity was achieved, confirming Mode 2 operates as a transparent learning switch with no restrictions.

6.2 Mode 1 — h1 Can Only Communicate with h2

With `MODE = 1`, packets from h1 to any host except h2 are dropped. The pingall output:

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> X h3 h4
h3 -> X h2 h4
h4 -> X h2 h3
*** Results: 25% dropped (9/12 received)
```

Figure 5: Mode 1 — pingall output showing 25% packet loss (9/12 received); h1 unreachable from h2/h3/h4

h1 successfully pings all hosts in the first round (destinations unknown, flooding used). Once MACs are learned, return packets from h2, h3, h4 back to h1 are blocked — shown as 'X'. 3 of 12 packets dropped (25%). The controller log confirms repeated MODE1 BLOCK entries for ports 3 and 4.

6.3 Mode 3 — Block All Traffic from h1

With `MODE = 3`, all packets where in_port == 1 are dropped. The pingall output:

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> X X X
h2 -> X h3 h4
h3 -> X h2 h4
h4 -> X h2 h3
*** Results: 50% dropped (6/12 received)
```

Figure 6: Mode 3 — pingall output showing 50% packet loss (6/12 received); h1 completely isolated

h1 is fully isolated — all pings from h1 fail (h1 → X X X). Since h1 never successfully transmits, its MAC is never learned by h2/h3/h4. Return pings to h1 also fail (ARP unanswered). 6 of 12 packets dropped (50%).

7. Comparative Summary

Mode	Policy	pingall Result	Packets Received
2	Allow All	All hosts reachable	**12/12 (0% dropped)**
1	h1 → h2 only	Return packets to h1 blocked	**9/12 (25% dropped)**
3	Block all from h1	h1 fully isolated	**6/12 (50% dropped)**

8. Observations and Analysis

8.1 MAC Learning Interaction with Mode Enforcement

The mode logic only fires when `dst_known == True`. This means h1's first ping to h3/h4 succeeds via flooding. Only after the MAC table is populated does the block take effect on return traffic — a subtle but important interaction visible in the Mode 1 results.

8.2 Mode 3 Total Isolation Effect

Blocking h1 at ingress prevents its MAC from being broadcast. Since no packet from h1 is ever forwarded, h2/h3/h4 cannot resolve h1's MAC address. Any ping to h1's IP address results in an unanswered ARP, causing those pings to fail even before ICMP echo is sent.

8.3 Controller Log Validation

The POX terminal (Figure 3) displayed repeated `MODE1 BLOCK: h1 → port 3` and `MODE1 BLOCK: h1 → port 4` messages, confirming that the block logic was invoked correctly for every qualifying packet.

8.4 Regression Test

Reinstalling flow rules by restarting the controller produced identical routing behaviour, confirming that the static routing paths are deterministic and reproducible — satisfying the regression test objective.

9. Conclusion

This experiment successfully demonstrated static routing and selective access control using the POX SDN controller. Three modes were implemented and validated:

- Mode 2 confirmed full connectivity with 0% packet loss — validating MAC learning and forwarding.
- Mode 1 enforced unidirectional access control from h1, yielding 25% packet drops as expected.
- Mode 3 achieved complete isolation of h1, producing 50% packet loss across the topology.

The experiment validates that SDN controllers can enforce fine-grained, programmable traffic policies without modifying physical network hardware.